KARMA Supplementary Appendix

1 Prompt Catalogue

For each prompt, we report its identifier, execution stage, and condition of use.

Table 1: Prompt catalogue summary.

ID	Stage	Prompt	Condition of use
P1	Entity linking	Entity candidate disambiguation	Only when fallback entity search returns multiple DBpedia candidates for an unlinked mention.
P2	Routing	Schema-constrained property extraction	Only when deterministic keyword/schema matching is inconclusive; candidate properties are passed back to deterministic routing and ASK checks.
P3	Decomp.	Question decomposition	Used after the direct symbolic shortcut fails and decomposition is needed.
P4	Decomp.	Early stopping / accumulated facts	Used during decomposition to decide whether accumulated facts already answer the original question.
P5	SPARQL	SPARQL generation	Used for direct SPARQL and routed sub-question SPARQL.
P6	SPARQL	SPARQL repair	Used once when a generated SPARQL query errors or returns empty results.
P7	SPARQL	SPARQL result verbalization	Used to convert SPARQL result rows into a concise final answer.
P8	Text fallback	Whole-question answer prompt	Used when a whole question is routed to text or symbolic execution cannot answer.
P9	Text fallback	Sub-question answer prompt	Used when an individual decomposed sub-question is routed to text.
P10	Synthesis	Final answer synthesis	Used to synthesize the final answer from resolved sub-question facts.
J1	Judge	LLM-as-judge system and user prompts	Used only for evaluation records.

1.1 Runtime KARMA prompts

P1. Entity candidate disambiguation

```
Given this question: "{question}"
Which entity best matches the mention "{mention}"?

{candidate_list}

Reply with ONLY the number (1-{num_candidates}) of the best match, or "none" if no candidate fits.
```

P2. Schema-constrained property extraction

```
Given this question and DBpedia schema, which properties are needed to answer it?

{schema}

QUESTION: {question}

Output a comma-separated list of property names (no dbo:/dbp: prefix).
Only output properties listed above. For 2-hop paths (e.g., "director's birthdate"),
list both: director, birthDate
If no properties can help, output: NONE

Properties:
```

P3. Decomposition prompt

```
Decompose this multi-hop question into independent sub-questions.

QUESTION: {question}
ENTITIES: {entity_names}
SCHEMA SUMMARY: {schema_summary}
{type_hint}
Rules:
- Each sub-question should ask for ONE fact
- If a sub-question needs the answer from another, mark the dependency
- For comparison questions: extract the same property for each entity (parallel, no dependencies)
- For bridge questions: first find the intermediate entity, then ask about its property
- If the question is simple (single fact lookup), return exactly 1 sub-question with no dependencies

Return JSON:
{
  "sub_questions": ["sub-Q1 text", "sub-Q2 text", ...],
  "dependencies": [[dependent_idx, dependency_idx], ...]
}

Example - Bridge: "When was the director of Inception born?"
{
  "sub_questions": ["Who directed Inception?", "When was [Q0] born?"],
  "dependencies": [[1, 0]]
}

Example - Comparison: "Who was born first, the director of X or Y?"
{
  "sub_questions": ["Who directed X?", "Who directed Y?",
    "When was [Q0] born?", "When was [Q1] born?"],
  "dependencies": [[2, 0], [3, 1]]
}

Example - Simple: "Who directed Inception?"
{
  "sub_questions": ["Who directed Inception?"],
  "dependencies": []
}

Example - Nested relative clause (3-hop): "When was the region north of where X is located created?"
{
  "sub_questions": ["Where is X located?", "What region is immediately north of [Q0]?", "When was [Q1] created?"],
  "dependencies": [[1, 0], [2, 1]]
}

Example - Passive voice: "Who is played by the director of Film X in Film Y?"
{
  "sub_questions": ["Who directed Film X?", "What role did [Q0] play in Film Y?"],
  "dependencies": [[1, 0]]
}

Example - Possessive chain: "What is the capital of the country where X's creator was born?"
{
  "sub_questions": ["Who created X?", "In which country was [Q0] born?", "What is the capital of [Q1]?"],
  "dependencies": [[1, 0], [2, 1]]
}

Additional rules for complex questions:
- Preserve active/passive voice direction: "played BY X" -> ask "What did [Q0] play?" NOT "Who played [Q0]?"
- For possessive chains (X's Y's Z), decompose from innermost entity outward
- Each sub-question MUST retain qualifying context (where/who/that clauses) from the original

JSON only, no explanation:
```

P4. Early stopping prompt

```
We pre-parsed the question into a list of dependencies, and the dependencies are sorted in a topological order, below
     is the question, the information summary, and the decomposed dependencies:

Question: {question}

Available Information:
{info_summary}

Decomposed dependencies:
{dependencies}

Current dependency to be answered:
{current_dependency}

Please analyze the question and the information summary, and the decomposed dependencies, and answer the following
     questions:
Please analyze:
1. Can the question be answered completely with this information? (Yes/No)
2. Summarize our current understanding based on available information.
```

```
Please format your response as a JSON object with these keys:
- "can_answer": boolean
- "current_understanding": string
```

P5. SPARQL generation prompt

```
You are a SPARQL expert for DBpedia. Generate a query to answer the question.

{schema}
{known_facts_block}
QUESTION: {question}

RULES:
1. Use the EXACT entity URIs provided in the schema
2. For multi-hop queries, chain triple patterns using dbo: properties (they return URIs, not strings).
   Use the "2-Hop chains" section in the schema to identify valid chains.
   Example: if schema shows dbo:director -> Person: birthDate, write:
   ?film dbo:director ?person . ?person dbo:birthDate ?date .
3. Prefer dbo: properties for multi-hop joins; only use dbp: for single-hop leaf values
4. Always add LIMIT 100
5. For comparison questions (who/which was born first, etc.), return data for BOTH entities so results can be compared
6. If an entity URI contains special characters (apostrophes, commas, parentheses), use full IRI syntax <http://
     dbpedia.org/resource/...> instead of dbr: prefix
7. If known facts provide entity URIs, use those URIs directly in your query
8. Generate ONLY the SPARQL query, no explanation
```

P6. SPARQL repair prompt

```
Your previous SPARQL query returned {problem}.

Previous query:
{sparql}

Fix the query using the schema below.

{schema}

QUESTION: {question}

Generate ONLY the corrected SPARQL query:
```

P7. SPARQL result verbalization prompt

```
Given the question and SPARQL query results from DBpedia, provide the answer.
Base your answer ONLY on the SPARQL results shown below, not on your own knowledge.
For comparison questions: if data is missing for one of the entities being compared, say "could not determine" instead
      of guessing.

Question: {question}
SPARQL Results:
{results}

End your response with FINAL ANSWER: <your answer>
```

P8. Whole-question text fallback prompt

```
Answer this question using the sources below.

Question: {question}

{kg_facts}Wikipedia text:
{context}

Instructions:
- Knowledge graph facts (if present) are verified structured data -- trust them
- Use Wikipedia text to fill gaps not covered by KG facts
- Show brief step-by-step reasoning
- End with exactly: FINAL ANSWER: <answer>
```

P9. Sub-question text fallback prompt

```
Answer this sub-question using the sources below.

Sub-question: {question}

{kg_facts}Wikipedia text:
```

```
{context}

Instructions:
- Knowledge graph facts (if present) are verified structured data -- trust them
- Use Wikipedia text to fill gaps not covered by KG facts
- Give a brief, factual answer
- End with exactly: FINAL ANSWER: <answer>
```

P10. Final answer synthesis prompt

```
You must give ONLY the direct answer in the most concise way possible.
DO NOT explain or provide any additional context.
If the answer is a simple yes/no, just say "Yes." or "No."
If the answer is a name, just give the name.
If the answer is a date, just give the date.
If the answer is a number, just give the number.
If the answer requires a brief phrase, make it as concise as possible.

IMPORTANT -- the sub-questions below form a reasoning chain:
- Earlier sub-questions find intermediate entities; later ones use those entities.
- The LAST sub-question's answer is usually what the original question asks for.
- Do NOT return an intermediate entity when a later sub-question has resolved the final answer.
- If a sub-question returns a list, pick the item that satisfies the condition in the original question.
- For yes/no questions with multiple sub-questions, combine ALL sub-answers (both must be true for "yes").
- If any sub-question is marked [UNKNOWN], the chain is incomplete. Do NOT fabricate
  a final answer by skipping the gap -- respond with: FINAL ANSWER: unknown

[Inserted only when is_comparison=True]
When comparing countries or nationalities:
- If answers contain city or state names, identify which country they belong to
- Compare at the country level, not city/state level
- Historical states (East Germany, Ottoman Empire) map to their modern successor

Question: {question}

Information Summary:
{info_summary}

End your response with exactly: FINAL ANSWER: <your answer>
```

1.2 Evaluation judge prompt

J1. LLM-as-judge prompt

```
System:
You are a strict evaluation judge for a QA benchmark. Determine if two answers refer to the exact same entity or fact.
      Precision matters more than recall -- when in doubt, answer no.

User:
Determine if the predicted answer is semantically equivalent to the gold answer.

Question: {question}
Gold answer: {gold}
Predicted answer: {pred}

Matching rules (answer YES):
- Same entity/fact with different surface forms (e.g. "July 16, 2013" vs "2013-07-16")
- Gold contained in a more specific predicted answer (e.g. "Munich" vs "Nymphenburg Palace, Munich")
- Added title/honorific/disambiguator (e.g. "Henry III" vs "Henry III, Holy Roman Emperor")
- Full formal name of same person (e.g. "Giovanni Mario" vs "Giovanni Mario de Candia")
- Administrative region containing the exact gold location (e.g. gold "Theni" vs pred "Theni district")
- Unambiguous well-known city with or without country (e.g. gold "Florence, Italy" vs pred "Florence")
- Nationality adjective matching country name (e.g. gold "Italy" vs pred "Italian")

Rejection rules (answer NO):
- Wrong, empty, or completely different entity
- Predicted is broader/vaguer than gold (e.g. gold "Taipei Medical University Hospital" vs pred "Taipei")
- Year only when gold has full date (e.g. gold "December 25, 1757" vs pred "1757")
- Related but distinct entities (e.g. gold "Hollywood" vs pred "Los Angeles")
- Same first name but different person (e.g. gold "Paris" vs pred "Paris Hilton")
- Predicted lists multiple answers and gold is NOT clearly among them
- Completely different location even if in same region (e.g. gold "Theni" vs pred "Mallingapuram")

First, explain in ONE sentence why the answers match or don't.
Then write your verdict on a new line: VERDICT: yes or VERDICT: no
```

2 End-to-end execution trace

Table 2: Trace-style examples of KARMA execution regimes selected from judged evaluation traces. Each block follows the same trace presentation pattern: schema projection rows separate observed predicates or candidate paths from annotations, and coverage rows report the covered relations, uncovered relations, and next branch. Text2SPARQL rows omit shared prefixes and abbreviate DBpedia IRIs with `dbr:`.

Step	Operator call	Output / state
Route: *Decomp: all SPARQL* **Question:** "Who was born first out of Marco De Luigi and Fabio Maistro?"		
1	EntityLink(Q)	E_Q={e_1, e_2}; e_1=`dbr:Marco_de_Luigi`; e_2=`dbr:Fabio_Maistro`.
2	SchemaProject (E_Q, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; for each $e \in E_Q$, A_Q: class=`dbo:Person`; P_Q: {`dbo:birthDate`, ...}.
3	Covers ($\mathcal{S}_Q, Q$)	covered={*born* -> `dbo:birthDate` for both entities}; uncovered={*born first* -> no accepted whole-question comparison route}; next branch=**DECOMPOSE**.
4	Decompose(Q)	LLM decomposition; D=($\{q_1, q_2\}, \emptyset$), ordered as q_1, q_2; q_1="When was Marco De Luigi born?"; q_2="When was Fabio Maistro born?"; $M \leftarrow \emptyset$.
5	Ground(q_1, M)	$\hat{q}_1$="When was Marco De Luigi born?".
6	EntityLink($\hat{q}_1$)	E_{q_1}={e_1=`dbr:Marco_de_Luigi`}.
7	SchemaProject ($E_{q_1}, \hat{q}_1$)	$\mathcal{S}_{q_1} = \langle P_{q_1}, A_{q_1} \rangle$; A_{q_1}: class=`dbo:Person`; P_{q_1}: {`dbo:birthDate`, ...}.
8	Covers ($\mathcal{S}_{q_1}, \hat{q}_1$)	**true**; covered={*born* -> `dbo:birthDate`}; uncovered=$\emptyset$; next branch=**SPARQL**.

9	Text2SPARQL
	($\hat{q}_1, \mathcal{S}_{q_1}, \mathcal{G}$)
		`SELECT ?birthDate`
		`WHERE {`
		`dbr:Marco_de_Luigi dbo:birthDate ?birthDate .`
		`}`
		`LIMIT 100`
		R={1978-03-21}; $M[q_1] \leftarrow$ *March 21, 1978*.

10	Ground(q_2, M)	$\hat{q}_2$="When was Fabio Maistro born?".
11	EntityLink($\hat{q}_2$)	E_{q_2}={e_2=`dbr:Fabio_Maistro`}.
12	SchemaProject ($E_{q_2}, \hat{q}_2$)	$\mathcal{S}_{q_2} = \langle P_{q_2}, A_{q_2} \rangle$; A_{q_2}: class=`dbo:Person`; P_{q_2}: {`dbo:birthDate`, ...}.
13	Covers ($\mathcal{S}_{q_2}, \hat{q}_2$)	**true**; covered={*born* -> `dbo:birthDate`}; uncovered=$\emptyset$; next branch=**SPARQL**.

14	Text2SPARQL
	($\hat{q}_2, \mathcal{S}_{q_2}, \mathcal{G}$)
		`SELECT ?birthDate`
		`WHERE {`
		`dbr:Fabio_Maistro dbo:birthDate ?birthDate .`
		`}`
		`LIMIT 100`
		R={1998-04-05}; $M[q_2] \leftarrow$ *April 5, 1998*.

15	Synthesize(Q, M)	LLM synthesis compares $M[q_1]$ and $M[q_2]$; a=*Marco De Luigi*.
Route: *Decomp: SPARQL+text* **Question:** "Where was the director of film *Age-Old Friends* born?"		
1	EntityLink(Q)	E_Q={e_0=`dbr:Age-Old_Friends`}.
2	SchemaProject (E_Q, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; A_Q: class=`dbo:Film`; P_Q: {`dbo:director`, ...}; candidate bridge path: `dbo:director`→`dbo:birthPlace`.
3	Covers ($\mathcal{S}_Q, Q$)	**false**; covered={*director* -> `dbo:director`}; uncovered={*born* -> unavailable bridge `dbo:director` → `dbo:birthPlace`}; next branch=**DECOMPOSE**.
4	Decompose(Q)	LLM decomposition; D=($\{q_1, q_2\}, q_1 \rightarrow q_2$); q_1="Who directed Age-Old Friends?"; q_2="Where was [q_1] born?"; $M \leftarrow \emptyset$.
5	Ground(q_1, M)	$\hat{q}_1$="Who directed Age-Old Friends?".
6	EntityLink($\hat{q}_1$)	E_{q_1}={e_0}.
7	SchemaProject ($E_{q_1}, \hat{q}_1$)	$\mathcal{S}_{q_1} = \langle P_{q_1}, A_{q_1} \rangle$; A_{q_1}: class=`dbo:Film`; P_{q_1}: {`dbo:director`, ...}.
8	Covers ($\mathcal{S}_{q_1}, \hat{q}_1$)	**true**; covered={*directed* -> `dbo:director`}; uncovered=$\emptyset$; next branch=**SPARQL**.

continued on next page

Table 2: Trace-style examples of KARMA execution regimes (continued).

Step	Operator call	Output / state

9	TEXT2SPARQL $(\hat{q}_1, \mathcal{S}_{q_1}, \mathcal{G})$	`SELECT ?director`
		`WHERE {`
		`dbr:Age-Old_Friends dbo:director ?director .`
		`}`
		`LIMIT 100`
		R={`dbr:Allan_Kroeker`}; $M[q_1] \leftarrow$ *Allan Kroeker*.

10	GROUND(q_2, M)	$\hat{q}_2$="Where was Allan Kroeker born?".
11	ENTITYLINK$(\hat{q}_2)$	E_{q_2}={e_1=`dbr:Allan_Kroeker`}.
12	SCHEMAPROJECT $(E_{q_2}, \hat{q}_2)$	$\mathcal{S}_{q_2} = \langle P_{q_2}, A_{q_2} \rangle$; A_{q_2}: class=`dbo:Person`; P_{q_2} does not contain `dbo:birthPlace`.
13	COVERS $(\mathcal{S}_{q_2}, \hat{q}_2)$	**false**; covered=$\emptyset$; uncovered={*born* -> no coverage-compatible `dbo:birthPlace`}; next branch=**TEXT**.
14	TEXTRAG $(\hat{q}_2, E_{q_2}, \mathcal{T})$	retrieved text: "Allan Kroeker (born February 29, 1960 in ***Winnipeg, Manitoba***) is a Canadian film and television director ..." $\rightarrow a_{q_2}$=*Winnipeg, Manitoba*; $M[q_2] \leftarrow$ *Winnipeg, Manitoba*.
15	SYNTHESIZE(Q, M)	LLM synthesis; a=*Winnipeg, Manitoba*.

continued on next page

Table 2: Trace-style examples of KARMA execution regimes (continued).

Step	Operator call	Output / state
Route: *Text only* **Question:** "Why did John Middleton Murry's wife die?"		
1	EntityLink(Q)	E_Q={e_0=`dbr:John_Middleton_Murry`}.
2	SchemaProject (E_Q, Q)	$\mathcal{S}_Q = \langle P_Q, A_Q \rangle$; A_Q: class=`dbo:Person`; P_Q contains neither `dbo:spouse` nor `dbo:deathCause`.
3	Covers ($\mathcal{S}_Q, Q$)	covered=$\emptyset$; uncovered={*wife* -> no observed `dbo:spouse`, *die* -> no observed `dbo:deathCause`}; next branch=**DECOMPOSE**.
4	Decompose(Q)	LLM decomposition; D=($\{q_1, q_2\}, q_1 \rightarrow q_2$); q_1="Who was John Middleton Murry's wife?"; q_2="Why did [q_1] die?"; $M \leftarrow \emptyset$.
5	Ground(q_1, M)	$\hat{q}_1$="Who was John Middleton Murry's wife?".
6	EntityLink($\hat{q}_1$)	E_{q_1}={e_0}.
7	SchemaProject ($E_{q_1}, \hat{q}_1$)	$\mathcal{S}_{q_1} = \langle P_{q_1}, A_{q_1} \rangle$; A_{q_1}: class=`dbo:Person`; P_{q_1} does not contain `dbo:spouse`.
8	Covers ($\mathcal{S}_{q_1}, \hat{q}_1$)	**false**; covered=$\emptyset$; uncovered={*wife* -> no coverage-compatible `dbo:spouse`}; next branch=**TEXT**.
9	TextRAG ($\hat{q}_1, E_{q_1}, \mathcal{T}$)	retrieved text: "... Murry is best remembered for his association with ***Katherine Mansfield***, whom he married in 1918 as her second husband ..." $\rightarrow a_{q_1}$=*Katherine Mansfield*; $M[q_1] \leftarrow$*Katherine Mansfield*.
10	Ground(q_2, M)	$\hat{q}_2$="Why did Katherine Mansfield die?".
11	EntityLink($\hat{q}_2$)	E_{q_2}={e_1=`dbr:Katherine_Mansfield`}.
12	SchemaProject ($E_{q_2}, \hat{q}_2$)	$\mathcal{S}_{q_2} = \langle P_{q_2}, A_{q_2} \rangle$; A_{q_2}: class=`dbo:Person`; P_{q_2} does not contain `dbo:deathCause`.
13	Covers ($\mathcal{S}_{q_2}, \hat{q}_2$)	**false**; covered=$\emptyset$; uncovered={*die* -> no coverage-compatible `dbo:deathCause`}; next branch=**TEXT**.
14	TextRAG ($\hat{q}_2, E_{q_2}, \mathcal{T}$)	retrieved text: "Mansfield was diagnosed with extrapulmonary ***tuberculosis*** in 1917; the disease claimed her life at the age of 34." $\rightarrow a_{q_2}$=*tuberculosis*; $M[q_2] \leftarrow$*tuberculosis*.
15	Synthesize(Q, M)	LLM synthesis; a=*Tuberculosis*.